

Global Min Cut
problem

Global min cut

$$G = (V, E)$$

Cut of G .

$$\text{Cut}(A, B)$$

Robustness.

Difference between mincut
and s - t cut.

$\text{Size}(A, B) \Rightarrow$ no. of edges
with A has one end
and other end
connected in B .

What is min cut?

- In a directed graph, you can **make one cut** in the graph
- **cut** is a partitioning the given graph into **two sets** so the **first partition** will be holding some set of vertices and all those vertices will belongs to two sets of nodes, one is source node and the other is target node
- The **first partition** will be holding some set of vertices and all those vertices will belong to the source node and another set of vertices (**second partition**) which belong to target node

What is min cut? (cont..)

- **Source node** is the number of **outflows** that we can **get** from the source node and the number of **inflows** that we can **give it** to target node
- If we look at this in the **s-t cut** problem, if you cut in the **middle** of the graph and obviously it will be our directed graph, we are just making two partitions of the same graph
- So, the **first partition** will consist of set of vertices that will belong to source node and the **second partition** will be the set of vertices that will belong to the target node
- Then find the **capacity value of those cut** and we need to **sum all the capacity values** of different number of edges which **the cut that has been enabled** on that particular edges
- If there is **multiple cuts**, then find out the **capacity** of those **multiple cuts** respectively

Difference between Min cut and s-t cut

Min cut	s-t cut
Graph is undirected	Directed graph is used
Doesn't have 2 separate nodes as Source and Target. i.e. knows only the vertices in the given graph	Has 2 separate nodes as Source and Target
On a cut in the graph, the size(A,B) is the no. of edges with one end that has been connected with A and the other end that has been connected with B Cut(A,B) : Your graph will consist of two partitions : Partition A and Partition B. Partition A will contain a specific subset of vertices, while Partition B will hold the remaining vertices.	
Mostly we will draw the cut in the anywhere of the graph	Mostly we will draw the cut in the middle of the graph

What is actually the min cut?

- we should have only the **minimum size**. Then how many no. of edges we have?
- if you have only the minimum number of edges like two or one in the graph and that we can call it as a minimum cut

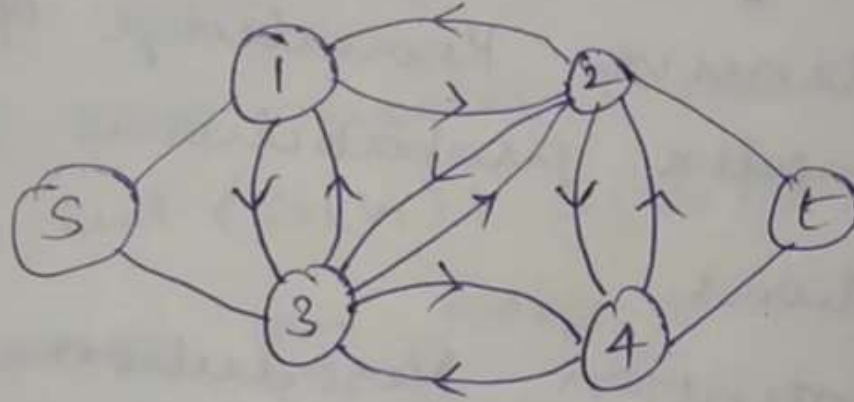
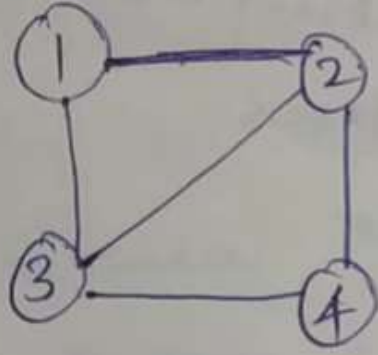
What is global minimum cut?

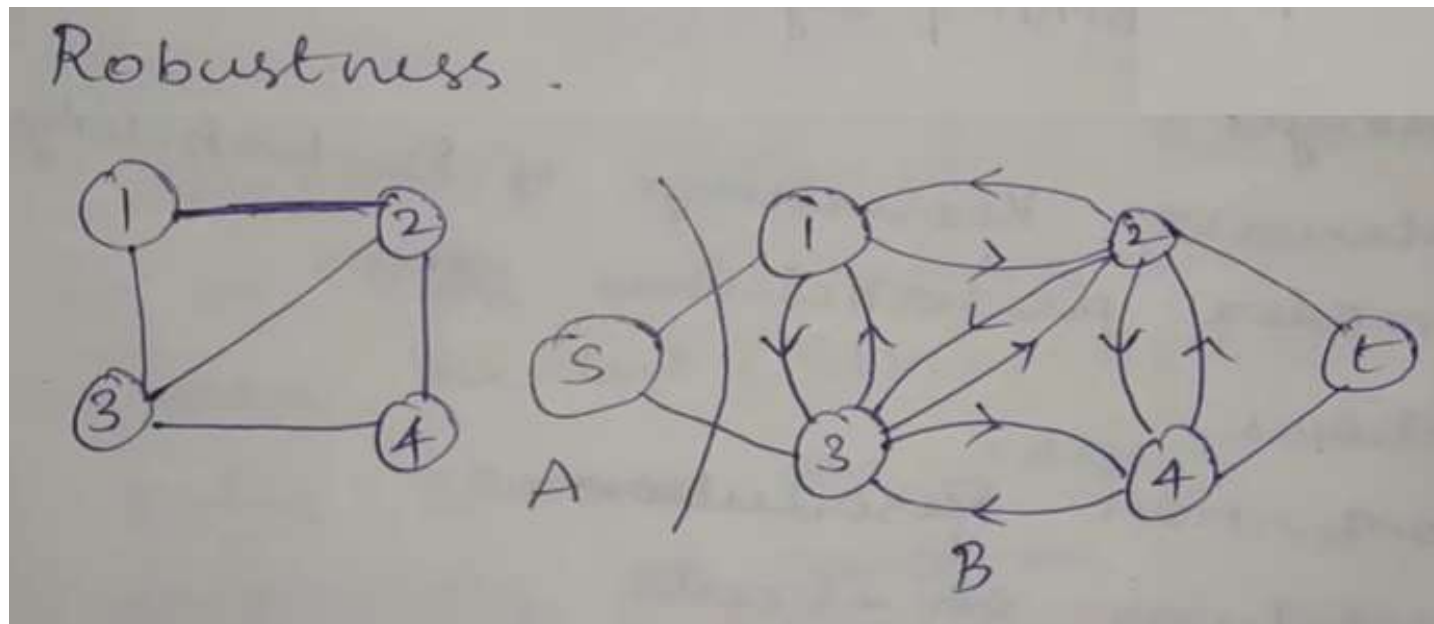
- Has the property of “Robustness”
- Robustness: it will have only the **smaller number of edges** in the graph
- We can have the cut anywhere in the graph

Global minimum cut?

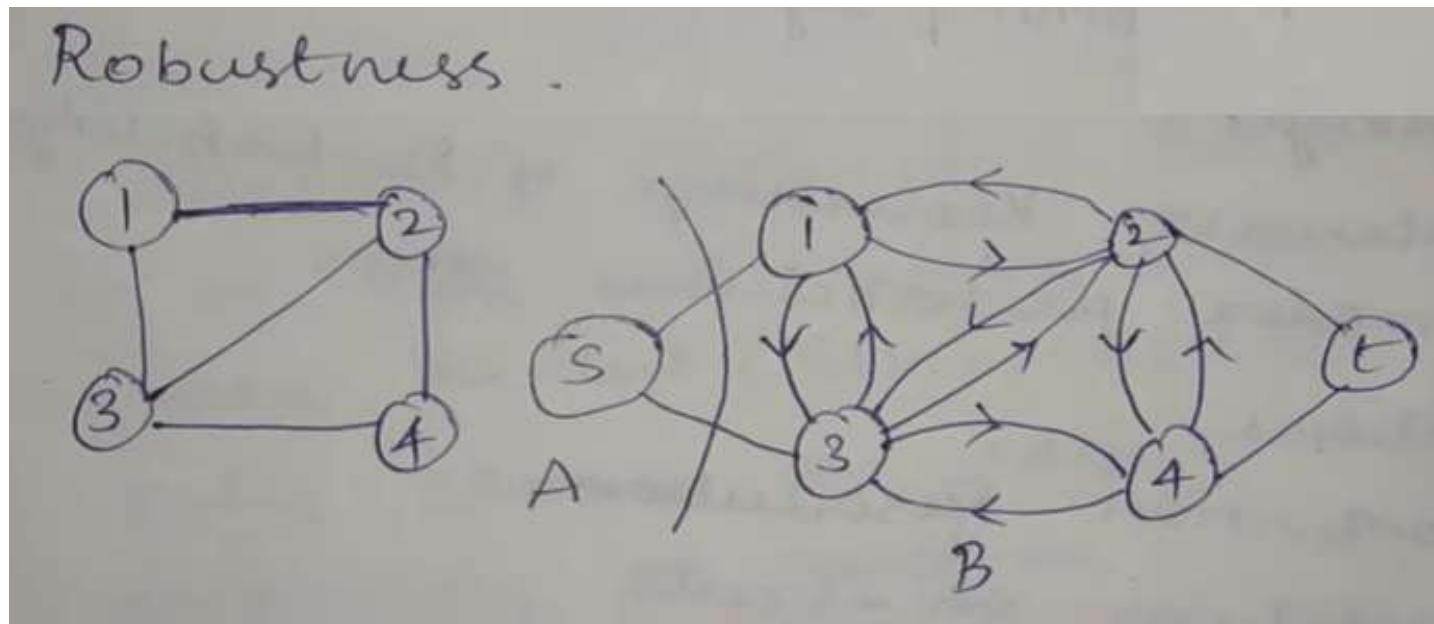
- In Global min cut, we have only undirected graph
- Step 1: Change this graph into a directed graph (i.e. changing undirected edge to a directed edge), then it will become multi-graph where multiple edges are present for same set of nodes.
 - i.e. for same pair of vertices, if we have more than one edge, then it is multi-graph
 - Include source node and target node to multigraph

Robustness .

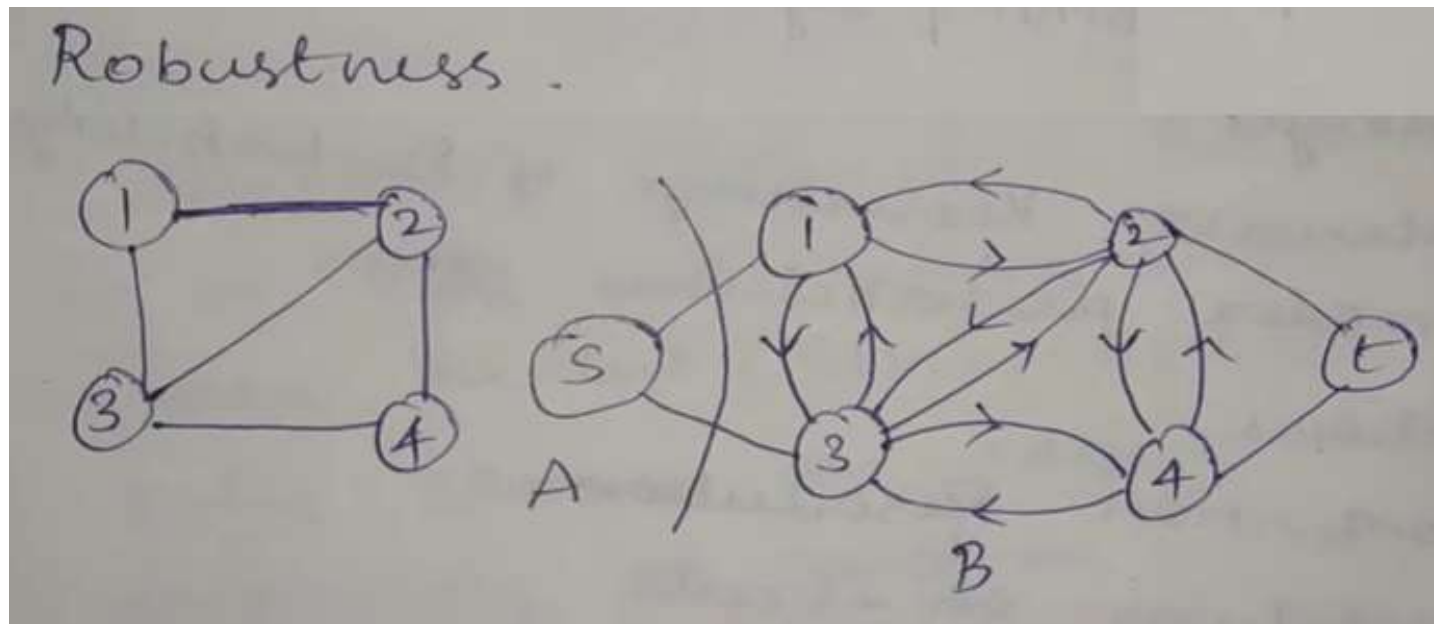




- If we follow one cut,
 - $A = \{S\}$
 - $B = \{1, 2, 3, 4, t\}$
- It is said to be **(n-1) directed edge min cut** problem or the graph
- In this way, we can cut all edges and we **need to find no. of edges** that it has on the cut



- If it is a **min cut**, then it is a **one solution**
- Need to recursively perform all these nodes until the graph holds the minimum no. of edges.
- This is said to be the **randomized approach** because **wherever** we want, we are **making this cut** in the graph
- It is not mandatory that the cut should be in either middle or left or right.



- **First**, we identify this cut, which minimizes the cut size by limiting it to only two edges. **Next**, we need to make another cut in the subsequent section of the graph, again aiming to minimize the number of edges in this cut. This process, focused on achieving minimal cut sizes, can be quite time-consuming and requires considerable effort.
- Therefore, we will now move on to our next topic, the contraction algorithm.

Contraction Algorithm

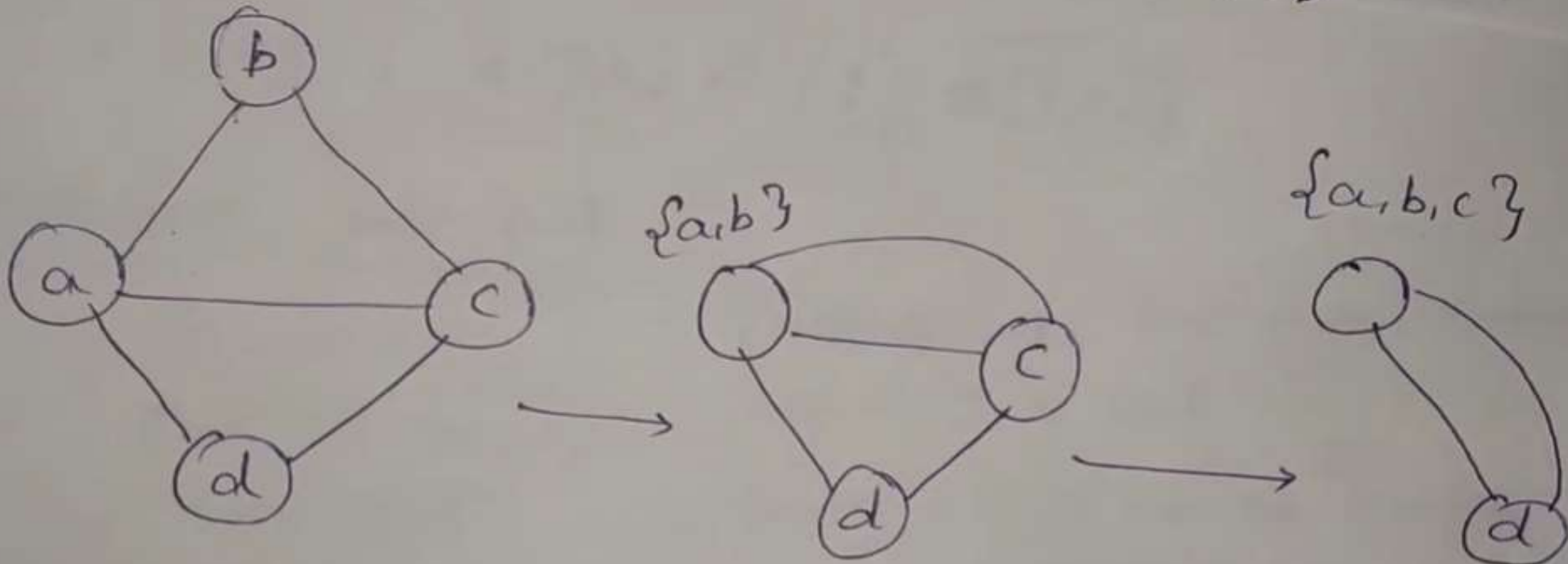
- We can solve this global min cut problem by the implementation of Randomization

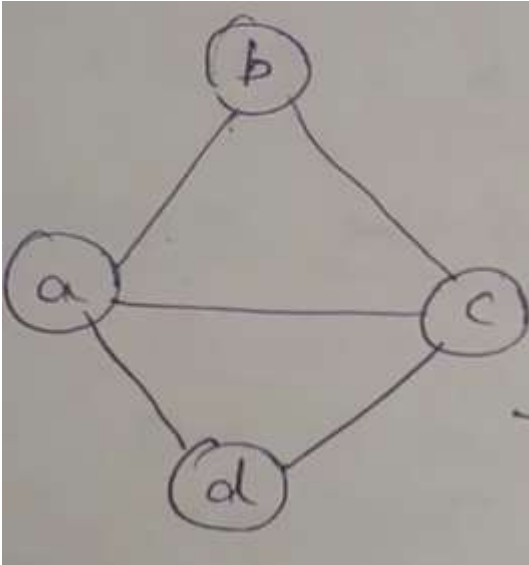
MultiGraph $G=(V,E)$
Multiple Parallel edges between same
Pair of nodes.

Goal: To perform the contraction algorithm

MultiGraph $G=(V,E)$

Multiple Parallel edges between same Pair of nodes.

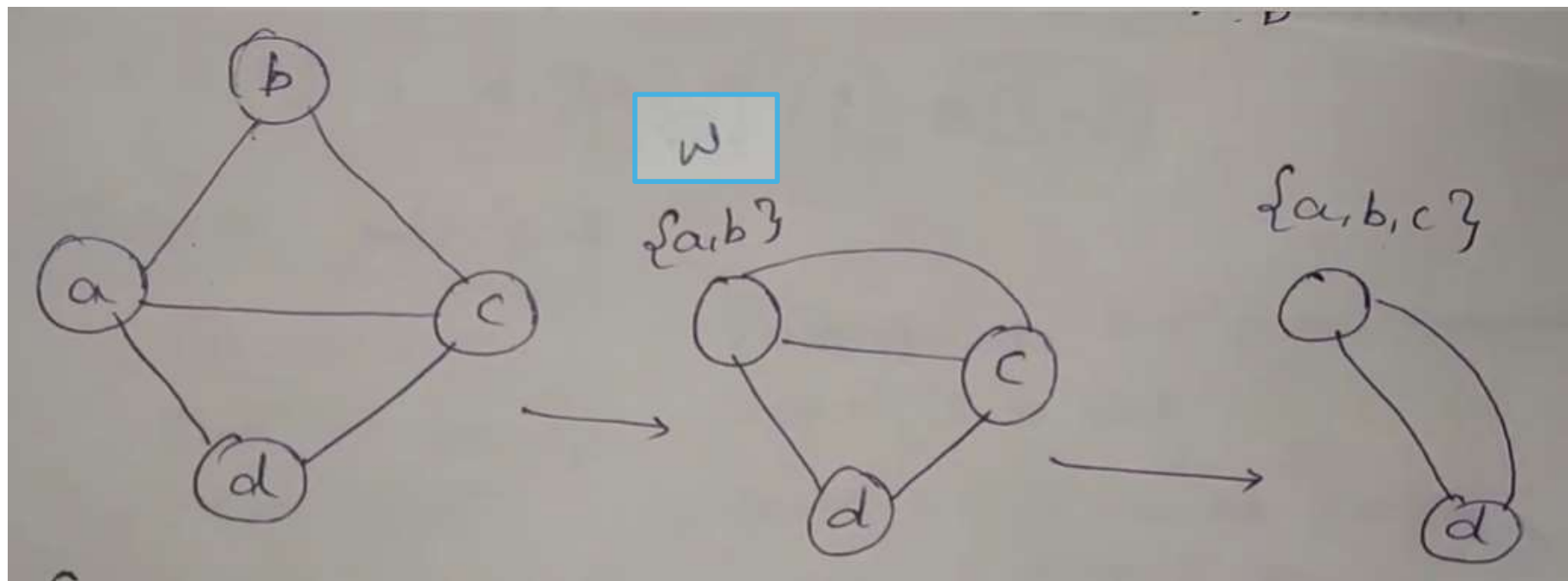




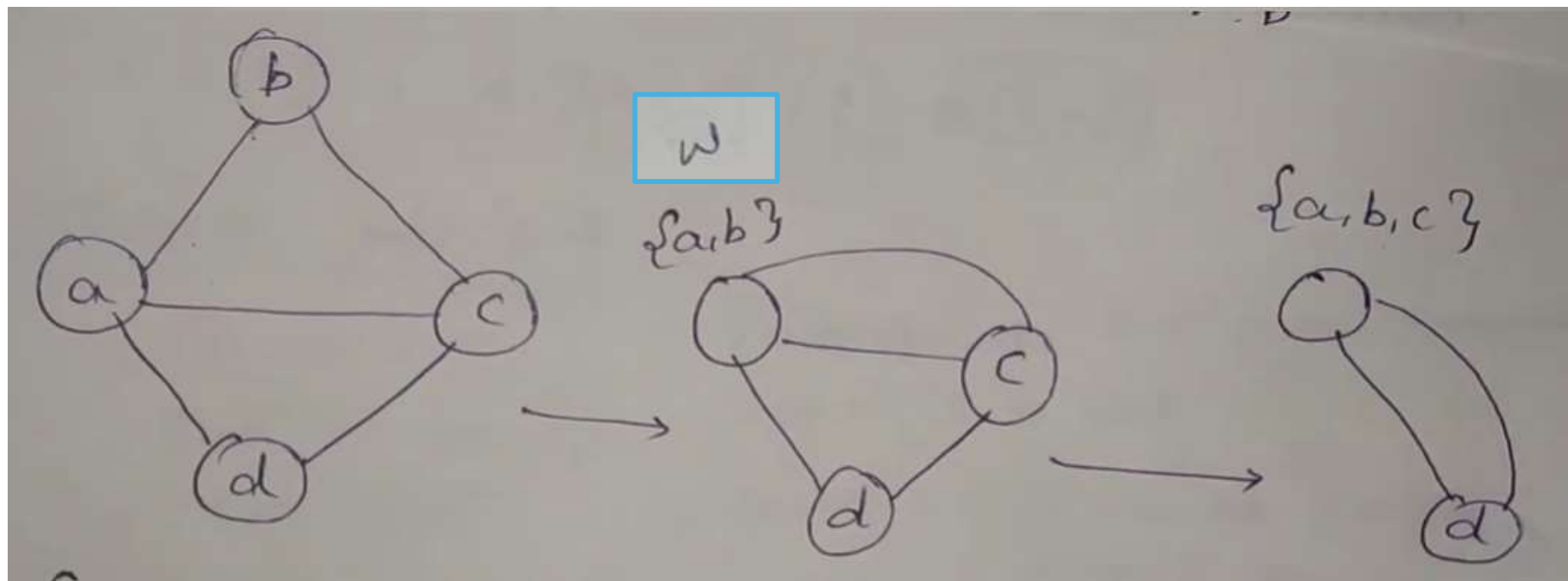
- Initially we have the multi graph
- It consists of 4 nodes: a, b, c and d
- To perform the **contraction**

algorithm:-

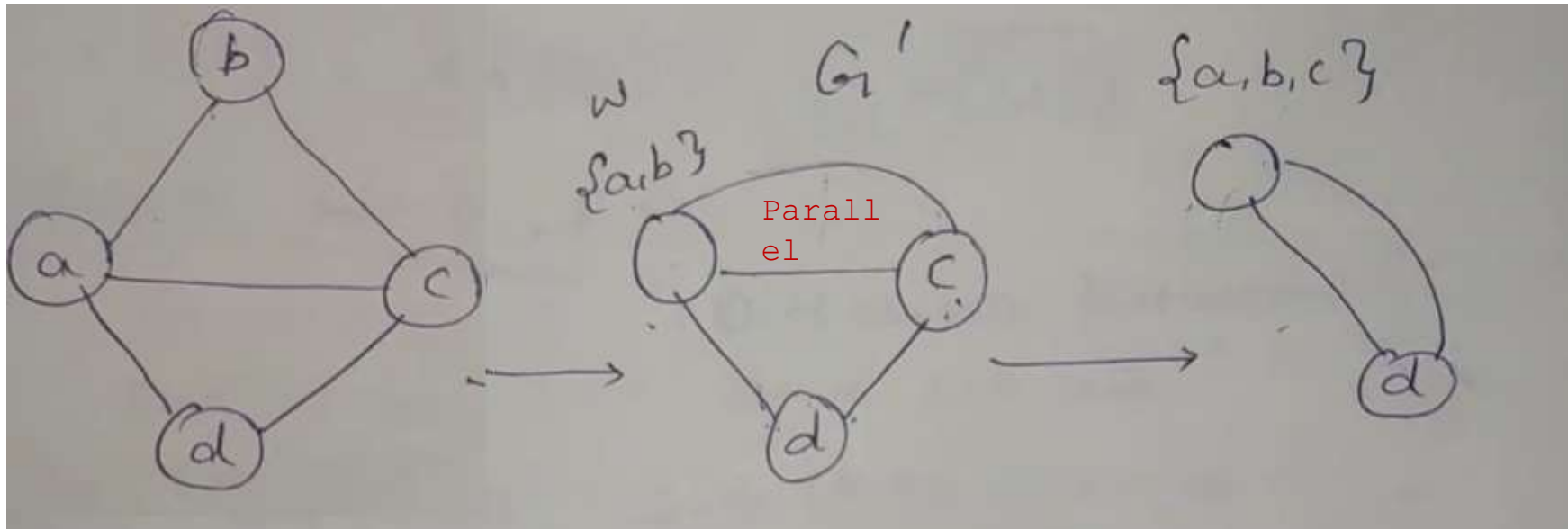
- Need to select any one of the edge from the graph.



- We **begin** by **selecting an edge** at **random**. For example, if we select the edge between nodes **a** and **b**, we then apply the **contraction method**.
- Contraction involves merging the two nodes, **a** and **b**, into a single node. Let's represent this combined node with the symbol **W**. Now, **W** represents both **a** and **b** as a single entity.
- Next, any duplicate or **self-loop edges** between **a** and **b** are removed from the graph. We keep other edges



- What is this node? It is (a,b) . It is representing it as w
- w will hold the set of vertices not a single vertex
- Again, we are **recursively** performing the contraction method on the graph now
- We start by **selecting an edge** at **random** and then **applying the contraction** algorithm. In contraction, we **merge** the **two connected nodes** into a **single node**, **eliminating the**



- If we randomly select an edge, we effectively choose one that **connects** two nodes with parallel paths. Let's say we select an edge between nodes $\{a,b\}$ and c . Since this forms a parallel structure with C , merging these nodes results in a single node labeled $\{a,b,c\}$.
- Now, we have two remaining edges connecting $\{a,b,c\}$ to another node $\{d\}$ —one originally from c and the other from $\{a,b\}$. **With these two edges**, we've reached a point where the graph has only two edges, achieving the minimum cut.
- At this stage, with only two edges remaining, we can **stop** the **contraction process**. This configuration is **our final solution**

Contraction Algorithm

- Contraction Algorithm applied to 4 node Multigraph $G=(V,E)$
- For each node V , we will record the set $S(v)$ of nodes that have been contracted into v
- Initially $S(v) = \{v\}$ for each v
- If G has two nodes v_1 and v_2 , then
 - return the cut $(S(v_1), S(v_2))$
- Else
 - Choose an edge $e = (u,v)$ of G uniformly at random
 - Let G' be the graph resulting from the contraction of e , with a new node w_{uv} (combination of any two nodes) replacing u and v
 - Define $S(W_{uv}) = S(u) \cup S(v)$
 - Apply the contraction algorithm recursively to G'
- End if

We perform the contraction algorithm until the graph contains only two vertices or two edges. This defines the global minimum cut problem.